Pembelajaran Mendalam
IF25 - 40401

Mgg2 : Training and Transfer Learning



Tim Penyusun: IEW, RIK, MCT, MCU

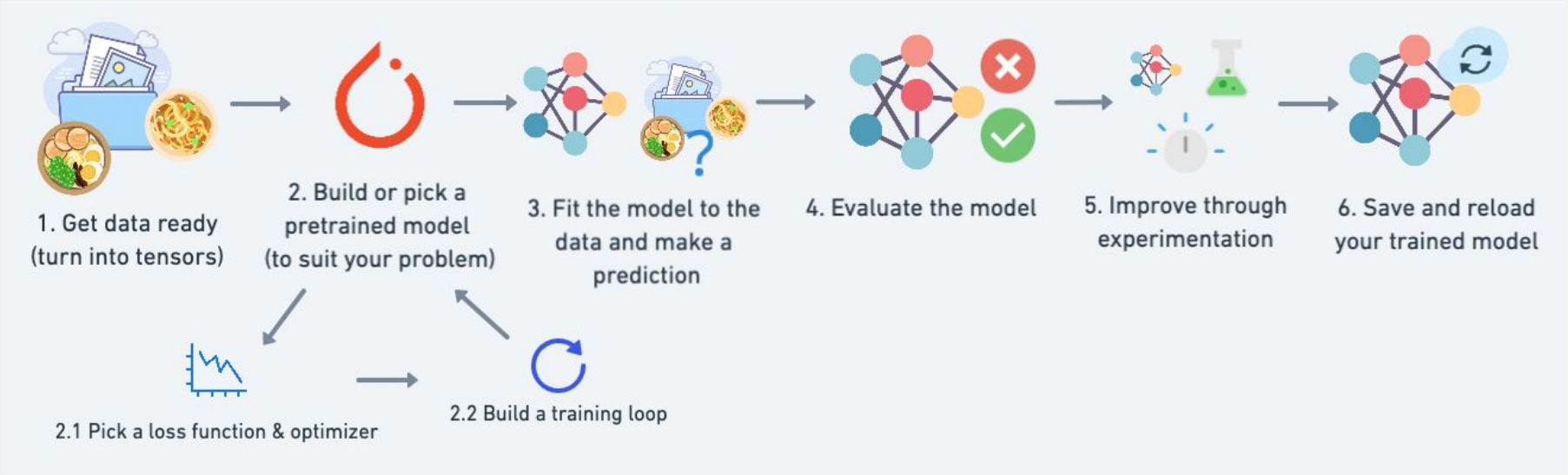
Semester Ganjil T.A. 2026-2027
Teknik Informatika, Fakultas Teknologi Industri
Institut Teknologi Sumatera

Pokok Bahasan

- Tahapan Dasar dari Pelatihan DL-Model
- *Scheduler & Normalization* (*Loss Value Convergence Strategy*)
- *Early Stopping & Regularization* (*Strategy to Prevent OverFitting*)
- *Transfer Learning Introduction (Zero-Shot Learning and Fine-Tuning)*

Tahapan Dasar dari Eksperimen DL

Tahapan dasar dari eksperimen deep learning sangat penting untuk dipahami karena proses pelatihan+validasi+pengujian dari model akan menggunakan tahapan dasar yang sama



Strategi pada *DL Model Training*

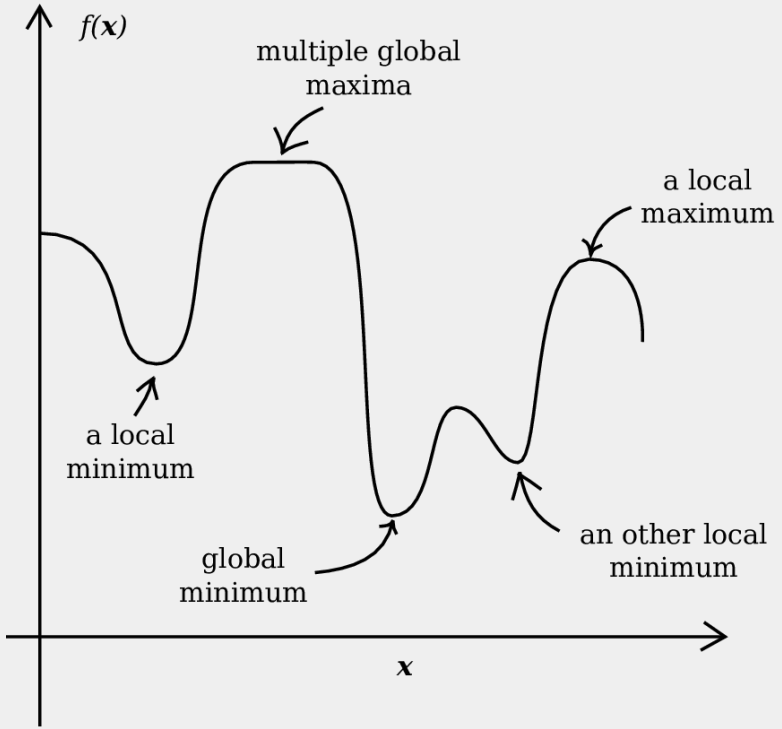
- Mempercepat Konvergensi dari *Loss Value* dengan *Scheduler* dan *Normalization*



Konvergensi Kemampuan Model

Convergence: parameter model (bobot dan bias) telah stabil dan tidak mengalami perubahan yang signifikan selama proses pelatihan, umumnya digambarkan dengan kondisi nilai loss yang tidak mengalami penurunan secara signifikan.

Tetapi, kondisi ini belum menjamin bahwa telah ditemukannya parameter model terbaik, karena kemungkinan kondisi ini juga bisa ditemui ketika proses pelatihan terjebak dalam **Local Optima**.



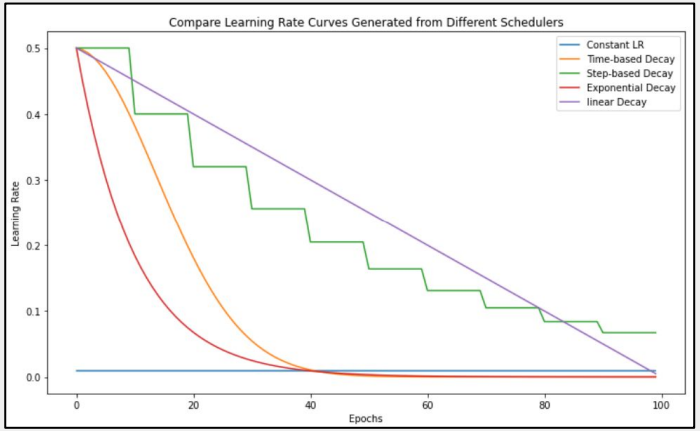
Scheduler: Konvergensi via Learning Rate

Selain optimizer yang memiliki tugas untuk pencarian kondisi model dengan Loss Value terkecil, terdapat scheduler (atau “Learning Rate Scheduler”) untuk membantu konvergensi.

Scheduler digunakan untuk menyesuaikan nilai learning rate secara dinamis selama proses pelatihan model.

Banyak jenis scheduler yang dapat digunakan pada Library Pytorch:

- `StepLR`: Menurunkan learning rate dengan nilai faktor tetap pada interval (epoch) yang ditentukan;
- `ExponentialLR`: Menurunkan learning rate dengan nilai faktor perkalian konstan pada setiap epoch.
- `ReduceLROnPlateau`: Mengurangi learning rate saat metrik yang dipantau (misalnya, validation loss) berhenti membaik selama jumlah epoch tertentu.
- dll...



Formulasi Matematis: Step Decay & Cosine Annealing

1. Step Decay Schedule

$$\eta_{\text{epoch}} = \eta_0 \times \gamma^{\lfloor \text{epoch}/s \rfloor}$$

- Mekanisme & Karakteristik: Menurunkan LR secara mendadak (step-like) setelah sejumlah epoch konstan terlampaui.
- Penjelasan Parameter:
  - η_0 : Learning rate awal (e.g. 0.1).
  - γ (Gamma) : Faktor multiplikasi penurunan (biasanya 0.1 s/d 0.5).
  - s (Step Size) : Interval epoch penurunan (e.g. setiap 30 epoch).
  - floor() : Fungsi pembulatan ke bawah.
- Kelemahan: Penurunan yang sangat drastis dan tiba-tiba terkadang mengganggu kestabilan adaptasi momentum optimizer.

2. Cosine Annealing (Tanpa Restarts)

$$\eta_t = \eta_{\min} + 0.5 \times (\eta_{\max} - \eta_{\min}) \times \left(1 + \cos\left(\frac{T_{\text{cur}} \times \pi}{T_{\max}}\right)\right)$$

- Mekanisme & Karakteristik: Menurunkan LR mengikuti lengkungan kurva kosinus yang sangat halus (smooth transition).
- Penjelasan Parameter:
  - η_{max} : Learning rate batas atas tertinggi (biasanya LR awal).
  - η_{min} : Batas bawah LR minimum (e.g. 1e-4 atau 1e-5) agar model tidak macet total.
  - T_{cur} : Epoch ke- berapa saat ini dalam training.
  - T_{max} : Total epoch target akhir siklus (e.g. 100).
- Keunggulan: Menghindari perubahan tajam yang disruptif pada parameter model, meng- hasilkan konvergensi akhir yang lebih rapi.

Akselerasi di Awal & Penurunan Reaktif di Plateau

3. Learning Rate Warm-up

- Mengapa diperlukan?
  - Di awal pelatihan (awal epoch), bobot model diinisialisasi acak. Gradien bisa berfluktuasi sangat ekstrem.
  - LR tinggi di awal dapat merusak bobot sebelum model menemukan orientasi arah optimasi yang benar.
- Mekanisme (Linear Warmup):
  - Memulai dengan LR mendekati nol, lalu merangkak naik secara bertahap menuju base LR dalam beberapa epoch awal (misal 5-10 epoch pertama).
  - Formula: $\eta t = \eta base \times (t / Twarmup)$ untuk $t <= Twarmup$
- Hasil: Menstabilkan fase training awal yang sangat sensitif sebelum akhirnya dilanjutkan dengan scheduler utama (misal Cosine).

4. ReduceLROnPlateau Scheduler

- Pendekatan Reaktif vs Preskriptif:
  - Scheduler lain (Step/Cosine) bersifat statis berdasar waktu (epoch-based).
  - ReduceLROnPlateau merespons kemajuan nyata performa model secara adaptif (performance-based).
- Prinsip Kerja:
  - Scheduler memantau metrik loss/akurasi pada dataset validasi (Validation Loss).
  - Jika Validation Loss stagnan dan tidak membaik selama sejumlah 'Patience' epoch yang ditentukan...
  - LR dikurangi secara otomatis sebesar faktor pengali (biasanya dikalikan 0.1 atau 0.5).
  - Parameter penting: mode ('min'/'max'), factor, patience, cooldown, threshold.

Penerapan Scheduler pada Pytorch

```
# Inisialisasi Optimizer & Scheduler
import torch
import torch.nn as nn
import torch.optim as optim

model = MyNeuralNetwork()
optimizer = optim.SGD(model.parameters(), lr=0.1)

# Pilihan 1: Step Decay Scheduler
scheduler = optim.lr_scheduler.StepLR(optimizer, step_size=30, gamma=0.5)

# Pilihan 2: Cosine Annealing
scheduler = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=100, eta_min=1e-4)

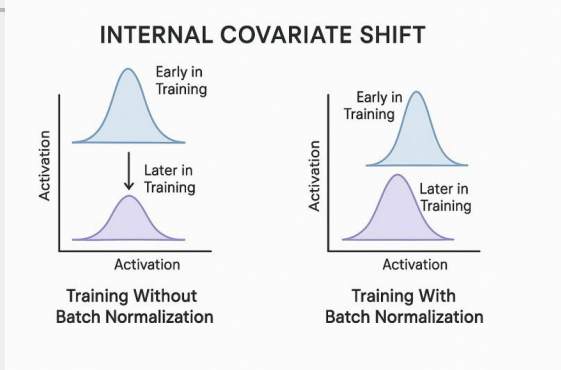
# Pilihan 3: Reduce LR on Plateau
scheduler = optim.lr_scheduler.ReduceLROnPlateau(optimizer, mode='min', factor=0.1, patience=5)

# Pilihan 4: Custom Linear Warmup + Cosine (SequentialLR)
# (Sering digunakan di arsitektur Transformer modern)
warmup_lr = optim.lr_scheduler.LinearLR(optimizer, start_factor=0.01, end_factor=1.0, total_iters=10)
cosine_lr = optim.lr_scheduler.CosineAnnealingLR(optimizer, T_max=90)
scheduler = optim.lr_scheduler.SequentialLR(optimizer, schedulers=[warmup_lr, cosine_lr], milestones=[10])
```

Normalization: Solusi Ketidakstabilan Layer Gradient

Masalah Utama: Internal Covariate Shift

Bayangkan Anda belajar melempar bola, tetapi setiap hari ukuran dan berat bola berubah drastis secara acak. Anda akan kesulitan untuk belajar secara konsisten.



Mekanisme dalam Jaringan Neural:

- Saat model berlatih, bobot di lapisan awal (e.g. Layer 1) diperbarui.
- Pembaruan ini menyebabkan distribusi output Layer 1 berubah secara konstan.
- Layer 2 menerima input yang terus berubah distribusinya setiap langkah iterasi. Hal ini disebut 'Internal Covariate Shift'.

Dampak:

- Lapisan terdalam terpaksa terus-menerus beradaptasi dengan distribusi input baru.
- Memperlambat proses pembelajaran secara drastis.

Normalization: Solusi Ketidakstabilan Layer Gradient

Solusi Radikal: Layer Normalizaton

Prinsip Operasi Normalisasi: Menstandarkan (standardize) output dari lapisan perantara secara paksa agar selalu memiliki rata-rata (mean) nol dan varians (variance) satu.

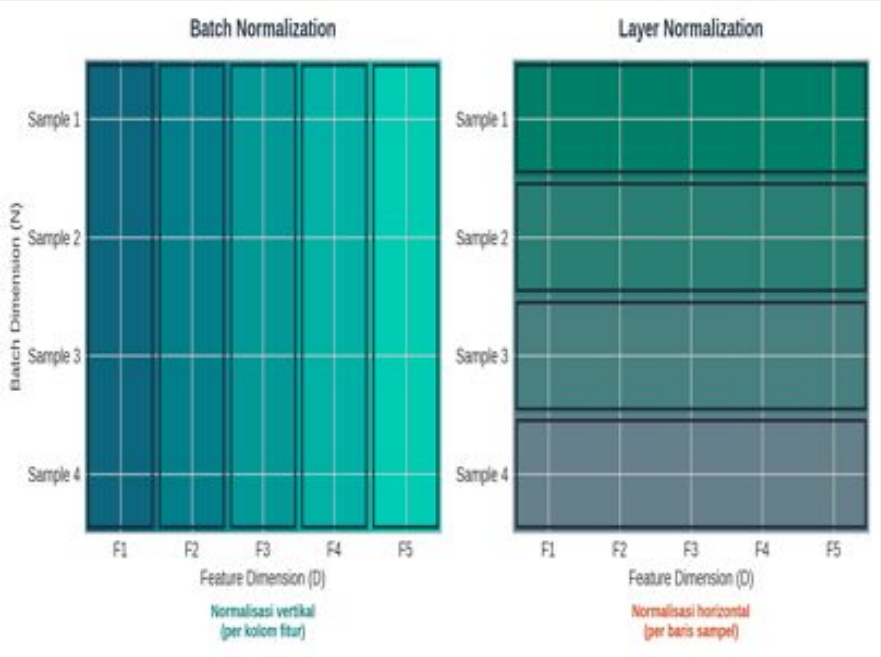
Mengapa Normalisasi Sangat Efektif?

1. Memangkas Vanishing/Exploding Gradients: Nilai input aktivasi dijaga di zona non-saturasi.
2. Memungkinkan Learning Rate Tinggi: Arsitektur menjadi sangat toleran terhadap langkah optimasi yang masif.
3. Mengurangi Sensitivitas Inisialisasi Bobot: Model tetap stabil meskipun diinisialisasi secara acak kurang ideal.
4. Efek Regularisasi Ringan: Perhitungan statistik pada mini-batch menambahkan kebisingan (noise) yang sehat.

BatchNorm vs LayerNorm

1. Formula Dasar Normalisasi (Sama):
   - Normalisasi: x_hat = (x - μ) / sqrt(σ^2 + ε)
   - Afinitas (Learnable): y = γ × x_hat + β

   (Di mana γ (scale) dan β (shift) dipelajari secara otomatis)
2. Batch Normalization (BN):
   - Statistik (μ, σ²) dihitung di seluruh sampel dalam satu mini-batch untuk setiap fitur/channel secara independen.
   - Sangat bergantung pada Batch Size. Jika batch size terlalu kecil (e.g. <4), estimasi statistik menjadi buruk dan training tidak stabil.
3. Layer Normalization (LN):
   - Statistik dihitung di seluruh fitur untuk setiap sampel individu secara mandiri.
   - Independen terhadap Batch Size. Sangat ideal untuk model berukuran besar, Sequence Models (RNN/Transformer), dan batch kecil.



Penempatan Lapisan: Mana yang Terbaik?

A. Pendekatan Klasik (Post-Activation):

Conv / Linear -> Activation -> BatchNorm -> Dropout

- Menormalisasi output yang sudah diaktifkan non-linear.
- Berfungsi dengan baik di arsitektur CNN klasik.

B. Pendekatan Modern (Pre-Activation):

Conv / Linear -> BatchNorm -> Activation -> Dropout

- Dinormalisasi persis SEBELUM masuk fungsi aktivasi (original paper recommendation).
- Terbukti menstabilkan gradien secara signifikan pada model yang sangat dalam (misalnya Deep ResNets).

Rekomendasi Utama: Lakukan eksperimen pada arsitektur Anda. Untuk model Transformer, LayerNorm selalu ditempatkan di pre-activation (Pre-LN Transformer).

Analisis Efek Terhadap Konvergensi

1. Kestabilan Landskap Optimasi (Loss Landscape):
   - Tanpa normalisasi, landskap loss sangat terdistorsi (tajam dan tidak beraturan).
   - Normalisasi memperhalus landskap loss (smooth landscape), membuat langkah gradien mengarah lurus ke lembah optimal.
2. Mengurangi Ketergantungan Inisialisasi:
   - Tanpa normalisasi, inisialisasi bobot yang meleset sedikit saja menyebabkan ledakan aktivasi atau vanishing gradient.
   - Dengan normalisasi, model sanggup konvergen dengan stabil meskipun diinisialisasi secara naif.
3. Laju Belajar Lebih Agresif:
   - Mempercepat konvergensi hingga 10x lipat lebih cepat karena bisa menggunakan Learning Rate yang besar di awal.

Normalisasi dalam Arsitektur PyTorch

```
# Aplikasi BatchNorm dan LayerNorm
import torch
import torch.nn as nn

# 1. Batch Normalization untuk MLP (1D)
# Input shape: (Batch Size, Features) -> (N, D)
bn1d = nn.BatchNorm1d(num_features=128)

# 2. Batch Normalization untuk CNN / Gambar (2D)
# Input shape: (Batch Size, Channels, Height, Width) -> (N, C, H, W)
bn2d = nn.BatchNorm2d(num_features=64)  # Dinormalisasi per channel

# 3. Layer Normalization untuk Sequence / Transformer
# Input shape: (Batch Size, Seq_Len, Embedding_Dim) -> (N, T, D)
ln = nn.LayerNorm(normalized_shape=256)  # Dinormalisasi per elemen token

# Contoh Integrasi dalam Arsitektur Model:
class DeepCNN(nn.Module):
  def __init__(self):
    super().__init__()
    self.conv = nn.Conv2d(3, 32, kernel_size=3, padding=1)
    self.bn = nn.BatchNorm2d(32)
    self.relu = nn.ReLU()

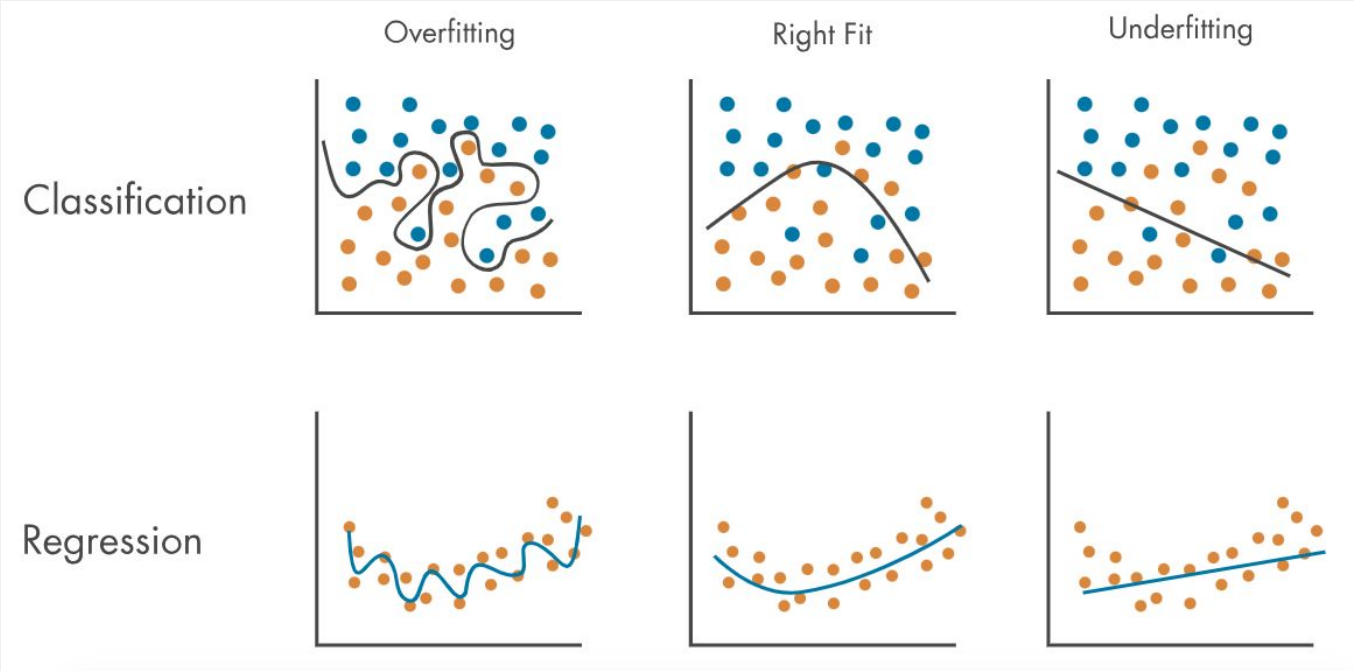
  def forward(self, x):
    # Struktur Pre-Activation modern
    return self.relu(self.bn(self.conv(x)))
```

Strategi pada *DL Model Training*

- Menghindari dari Kondisi *OverFitting* dengan *Early Stopping* dan *Regularization*

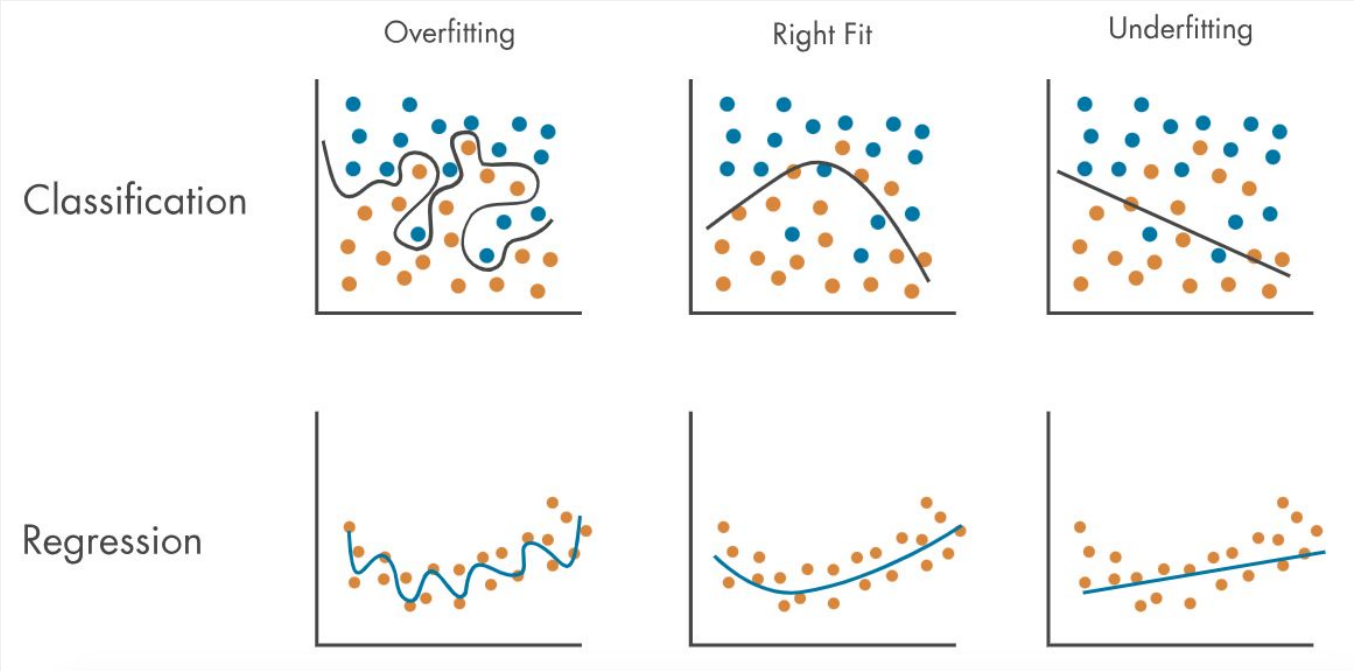


Overfitting v.s. Underfitting



Overfitting terjadi ketika suatu model berkinerja sangat baik pada *training data* tetapi buruk pada data baru yang belum pernah dilihat sebelumnya (*validation atau testing data*). Hal ini terjadi karena model tersebut telah mempelajari *noise* dan detail tertentu, dari pada pola yang mendasari sebaran datanya.

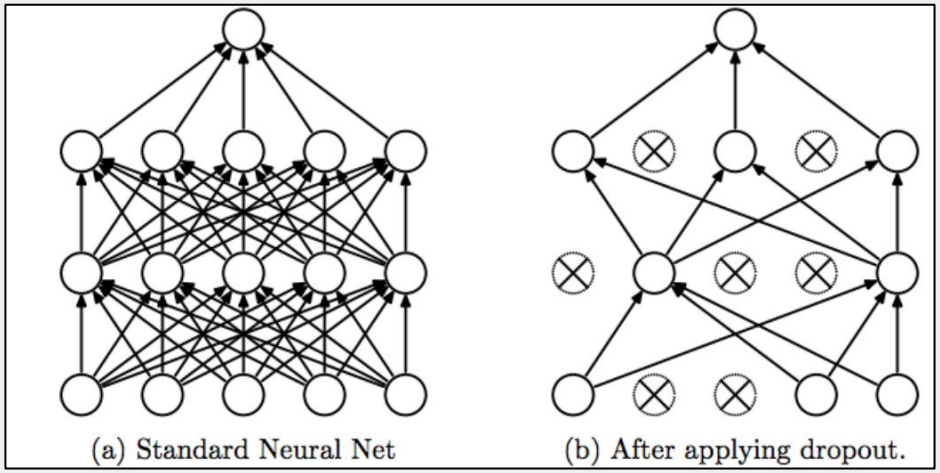
Overfitting v.s. Underfitting

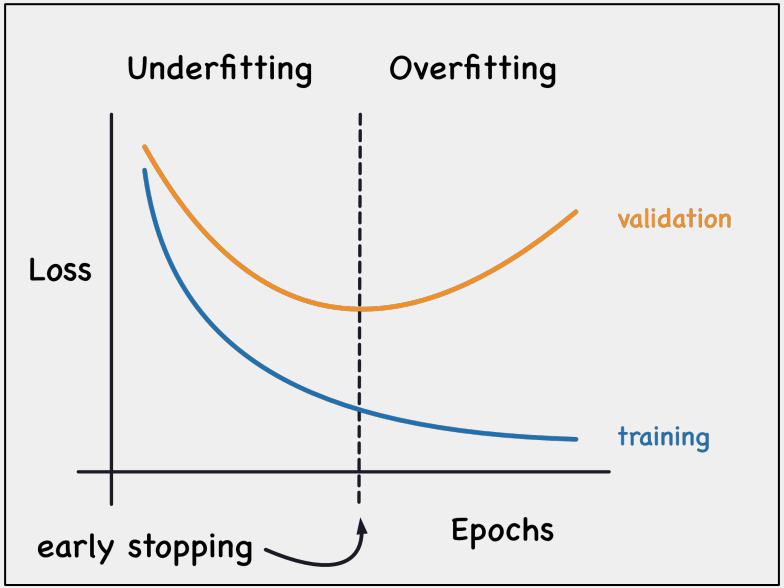


Sebaliknya, **underfitting** terjadi ketika model terlalu sederhana untuk menangkap pola penting dari data, sehingga menghasilkan kinerja yang buruk pada *training data* terlebih pada data baru (*validation* atau *testing data*).

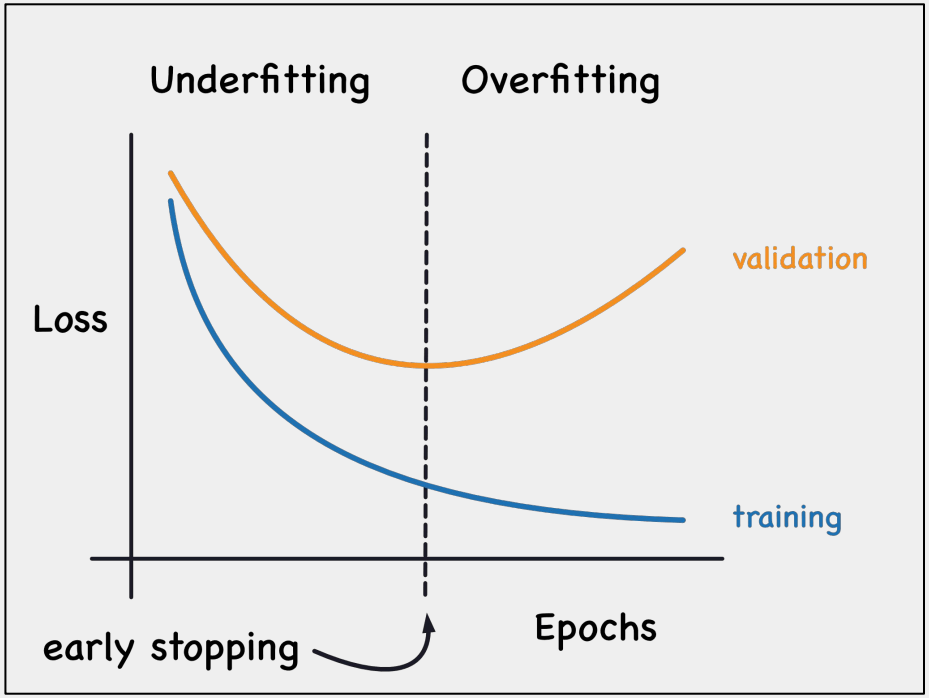
Dropout dan Early Stopping

Dua teknik dari beragam strategi yang bisa digunakan untuk menghindari model terjebak pada kondisi overfitting





Early Stopping



Prosedur **early stopping** berfokus untuk memantau performa model pada validation set terpisah selama pelatihan dan menghentikan proses pelatihan saat performa pada val set ini mulai menurun atau berhenti membaik selama sejumlah epoch tertentu.

Terdapat nilai "**patience**" sebagai jumlah toleransi penurunan kemampuan model pada validation set pada sejumlah epoch yang berurutan

Early Stopping: Penerapan di Pytorch

```
criterion = torch.nn.BCELoss()
optimizer = torch.optim.SGD(model.parameters(), lr=0.1)
patience_counter = 0
patience_limit = 5

model.train()
val_history = []
for epoch in range(epochs):
  # ... (training steps: forward pass, backward pass, optimizer.step()) ...

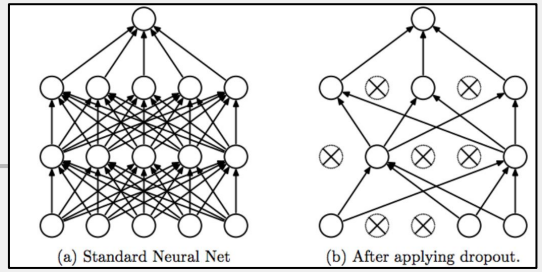
  current_validation_loss = calculate_validation_loss()

  if current_validation_loss < val_history[-1]:
    patience_counter = 0
  else:
    patience_counter += 1

  val_history.append(current_validation_loss)
  if patience_counter >= patience_limit:
    print("Early stopping triggered!")
    break
```

PyTorch

Dropout & Penerapannya



Dropout diterapkan dengan menonaktifkan secara acak sebagian neuron selama proses pelatihan pada setiap iterasi.

"Penonaktifan" acak yang dipaksakan ini membuat **jaringan lebih tangguh**, **mencegah neuron menjadi terlalu bergantung pada neuron lain yang spesifik**, dan **mendorongnya untuk mempelajari fitur yang lebih tangguh dan dapat digeneralisasi**.

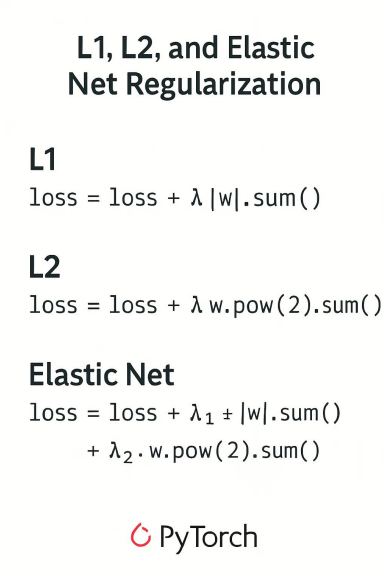
Selama pengujian, semua neuron akan digunakan kembali.

```
class SimpleANN(torch.nn.Module):
  def __init__(self, input_size, hidden_size1, hidden_size2, output_size):
    super(SimpleANN, self).__init__()
    self.fc1 = torch.nn.Linear(input_size, hidden_size1)
    self.fc2 = torch.nn.Linear(hidden_size1, hidden_size2)
    self.fc3 = torch.nn.Linear(hidden_size2, output_size)
    self.relu = torch.nn.ReLU()
    self.tanh = torch.nn.Tanh()
    self.sigmoid = torch.nn.Sigmoid()
    self.dropout = nn.Dropout(p=0.5)

  def forward(self, x):
    x = self.fc1(x)
    x = self.relu(x)
    x = self.dropout(x)
    x = self.fc2(x)
    x = self.tanh(x)
    x = self.dropout(x)
    x = self.fc3(x)
    x = self.sigmoid(x)
    return x
```

Penambahan Penalti pada Bobot Model

Regularization adalah semacam "hukuman" atau penalti yang diberikan kepada model agar dia tidak menghafal secara rumit, melainkan dipaksa mencari pola yang lebih sederhana. Semakin rumit cara model berpikir, semakin besar hukuman yang diberikan.



L1 Regularization (Lasso)

- Bekerja seperti seorang penganut gaya hidup minimalis. Ia berprinsip: "Jika sebuah informasi (fitur) tidak benar-benar penting, buang saja sama sekali."
- Efek Utama (Sparsity): L1 akan memaksa banyak bobot (weights) dari koneksi antar neuron menjadi tepat nol (0). Koneksi tersebut pada dasarnya diputus.

L2 Regularization (Ridge / Weight Decay)

- Bekerja seperti konsep kerja sama tim. L2 sangat membenci jika ada satu fitur (input) yang bertingkah seperti superstar dan terlalu mendominasi keputusan model. Ia memaksa semua fitur memberikan kontribusi secara merata.
- Efek Utama (Distribusi Bobot): L2 menekan semua bobot (weights) menjadi sangat kecil mendekati nol, tetapi tidak sampai benar-benar nol. Tidak ada koneksi yang dibuang, hanya dikecilkan pengaruhnya.

Data Augmentation (Computer Vision & NLP)

Definisi & Filosofi:

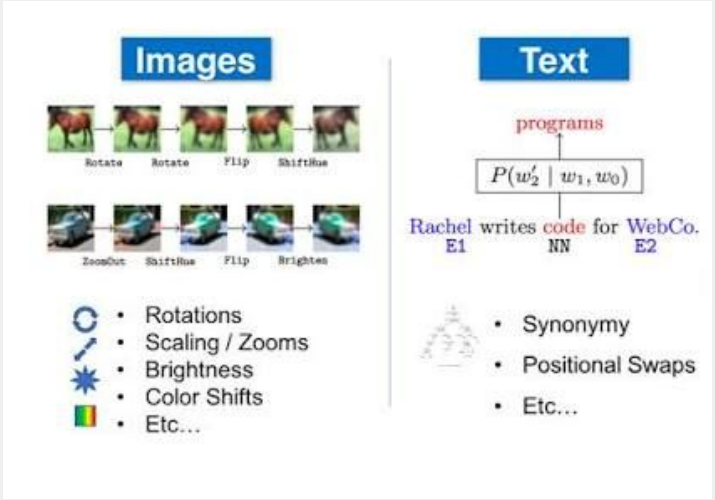
- Cara terbaik menghindari overfitting adalah memperbanyak data latih. Namun, mencari data baru sangatlah mahal.
- Augmentasi merekayasa sampel baru secara artifisial dengan memodifikasi data latih yang ada saat ini.

Teknik pada Data Gambar (Computer Vision):

- Geometri: `RandomRotation`, `RandomHorizontalFlip`, `RandomResizedCrop`.
- Warna: `ColorJitter` (kecerahan, kontras, saturasi).
- Noise: `RandomErasing`, `GaussianNoise`.

Keunggulan Utama:

- Memaksa model mengenali objek berdasarkan esensi bentuk fungsionalnya, bukan orientasi posisi pixel yang kaku.



Label Smoothing (Regulasi Target)

Masalah Hard One-Hot Label:

- Klasifikasi standar menggunakan target biner mutlak: y = [1.0, 0.0, 0.0] untuk Kelas 1.
- Untuk mencapai 100% probabilitas pada softmax, logit model terpaksa bernilai sangat besar (menuju tak terhingga). Ini menyebabkan model terlalu percaya diri (overconfident).



Formula Label Smoothing (K kelas, smoothing α):

- y_smoothed = y * (1 - α) + (α / K)
- Contoh (K=3, α=0.1): Target Kelas 1 yang semula [1, 0, 0] dihaluskan menjadi [0.933, 0.033, 0.033].

Efek Positif:

- Mencegah logit membesar tak terbatas selama training.
- Meningkatkan kualitas kalibrasi prediksi model dan ketahanan terhadap label data latih yang kotor (noisy).

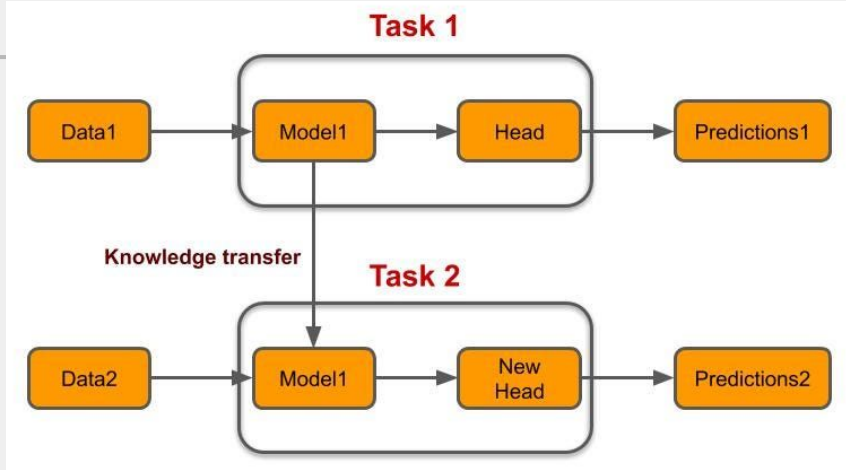
Pengenalan *Transfer Learning*

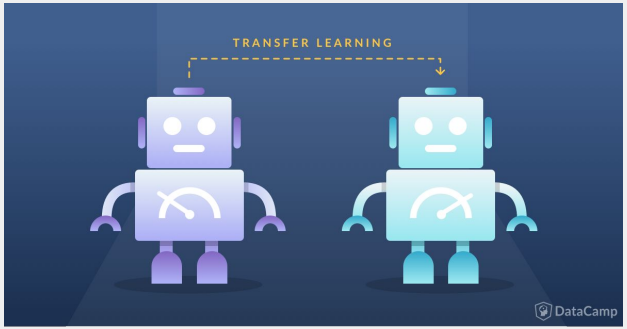
- Instance-Based
- Feature-Based
- Parameter-Based



Transfer Learning

- Pada penerapan teknis, sangat sedikit peneliti yang melakukan training pada keseluruhan arsitektur Deep Learning model dari awal (dengan inisialisasi bobot yang acak).
  - Hal ini dikarenakan, relatif jarang penelitian yang memiliki kumpulan data dengan ukuran yang memadai.
- Sebaliknya, beberapa peneliti umumnya melakukan training terhadap model terlebih dahulu dengan kumpulan data yang sangat besar.
  - Contoh ImageNet untuk Image Classification atau Object Detection yang berisi 1,2 juta gambar dengan 1000 class.
  - Selanjutnya model ConvNet yang sudah dilatih disesuaikan kembali untuk task yang diharapkan sebenarnya.





Transfer Learning: Notasi dan Definisi

- Notasi
  - Domain $\mathbb{D}=\{X, p(x)\}$
    - Ruang fitur X
    - Distribusi data p(x)
  - Task $\mathbb{T}=\{y, f(.)\}$
    - Ruang label y
    - Fungsi prediksi objektif $f(.)$

- Definisi
  - Diberikan **source domain $\mathbb{D}_S$** dengan mempertimbangkan **learning task $\mathbb{T}_S$** dan **target domain $\mathbb{D}_T$** dengan mempertimbangkan **learning task $\mathbb{T}_T$**
  - **Transfer Learning** merupakan proses untuk meningkatkan fungsi prediksi target $f_T(.)$ dengan menggunakan informasi yang berhubungan dari $\mathbb{D}_S$ dan $\mathbb{T}_S$, dimana $\mathbb{D}_S \neq \mathbb{D}_T$ atau $\mathbb{T}_S \neq \mathbb{T}_T$

Transfer Learning: Variasi

- $\mathbb{D}_S \neq \mathbb{D}_T$
  - $X_S \neq X_T$
    - Heterogeneous transfer learning
    - Ex: dua kumpulan dokumen dideskripsikan dalam bahasa yang berbeda
  - $P(X_S) \neq P(X_T)$
    - Adaptasi domain
    - Ex: dua kumpulan dokumen yang fokus pada topik berbeda
- $\mathbb{T}_S \neq \mathbb{T}_T$
  - $Y_S \neq Y_T$
    - Adaptasi pada nilai target data
    - Ex: source memiliki dua kelas (positif dan negatif), sedangkan target menambahkan satu kelas lain (netral)
  - $P_S(y|x) \neq P_T(y|x)$
    - Adaptasi pada atribut data
    - Suatu kata memiliki arti yang berbeda dalam dua domain

Transfer Learning

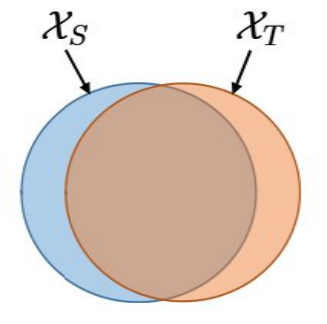
Beberapa jenis metode yang terdapat pada transfer learning:

- Instance based Transfer
  - Fokus pekerjaan yaitu pembobotan kembali (re-weight) data-data dari target set dengan mempertimbangkan pada source set
- Feature based Transfer
  - Fokus pekerjaan pada pemetaan fitur-fitur dari source dan target set dalam ruang fitur yang umum
- Parameter based Transfer
  - Fokus pekerjaan pada pelatihan model parameter dari target set dengan mempertimbangkan pada model parameter dari source set

(1) Instance-based Transfer Learning

- Asumsi umum
  - Domain source dan target memiliki **irisan (*overlapping*) domain fitur yang besar atau bahkan berbagi ruang fitur yang sama**.

$$X_S \simeq X_T$$



  - **Ruang dari label seharusnya sama**

$$y_S \simeq y_T$$

- Contoh penerapannya
  - Data rekam jejak kondisi alat elektronik pada bagian/unit kerja yang berbeda
  - Analisis sentimen pada topik yang berbeda

(1) Instance-based Transfer Learning

Case 1 : Domain Adaptation

- Problem Setting
  - Diberikan source domain labeled data $\mathcal{D}_S = \{x_{S_i}, y_{S_i}\}_{i=1}^{n_S}$ dan target domain data $\mathcal{D}_T = \{x_{T_i}\}_{i=1}^{n_T}$
  - Mempelajari f_T sedemikian hingga nilai loss pada target data adalah kecil
    $$\sum_i = \mathcal{L}(f_T(x_{T_i}), y_{T_i})$$
  - Dimana y_{T_i} tidak diketahui
- Asumsi
  - Label space yang sama $y_S = y_T$
  - Keterhubungan yang sama $p(y_S|x_S) = p(y_T|x_T)$
  - Ruang fitur yang hampir sama $X_S \simeq X_T$
  - Distribusi data yang berbeda $p_S(x) \neq p_T(x)$

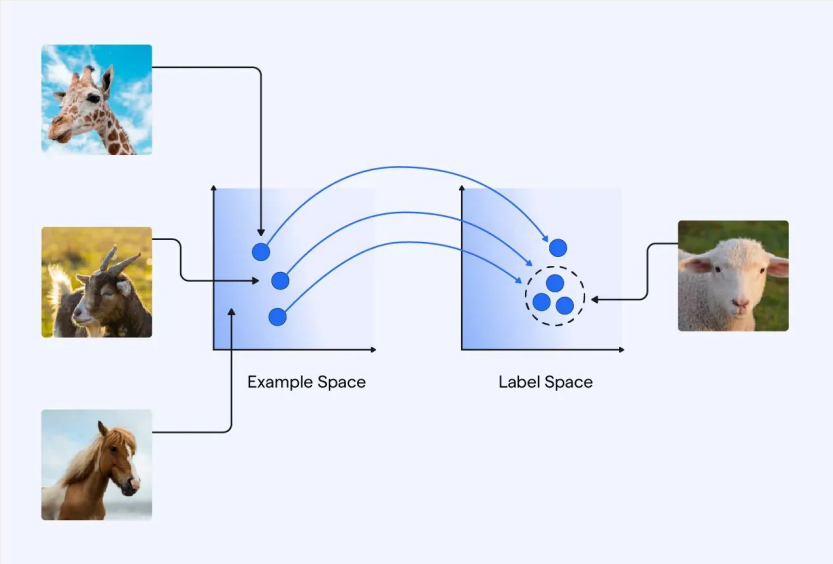
(1) Instance-based Transfer Learning

Case 2 : Labels Adaptation

- Problem Setting
  - Diberikan source domain labeled data $\mathcal{D}_S = \{x_{S_i}, y_{S_i}\}_{i=1}^{n_S}$
  - Dan juga target domain data yang terbatas $\mathcal{D}_T = \{x_{T_i}, y_{T_i}\}_{i=1}^{n_T}$
  - Mempelajari f_T sedemikian hingga nilai loss pada target data adalah kecil $\sum_i = \mathcal{L}(f_T(x_{T_i}), y_{T_i})$
- Asumsi
  - Label space yang sama $y_S = y_T$
  - Keterhubungan yang berbeda $p(y_S|x_S) \neq p(y_T|x_T)$
  - Ruang fitur yang hampir sama $X_S \simeq X_T$
  - Distribusi data yang berbeda $p_S(x) \neq p_T(x)$

(1) Instance-based Transfer Learning

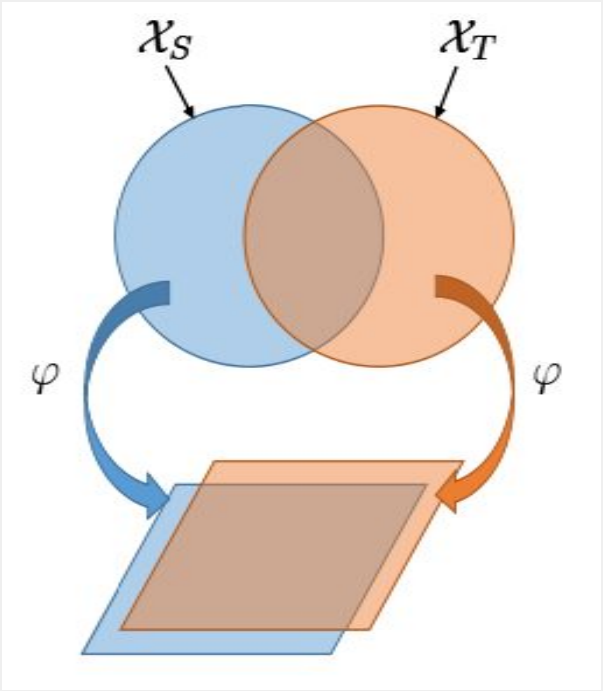
- Contoh metode yang berbasis pada Instance Transfer Learning
  - Zero Shot Learning



(2) Feature-based Transfer Learning

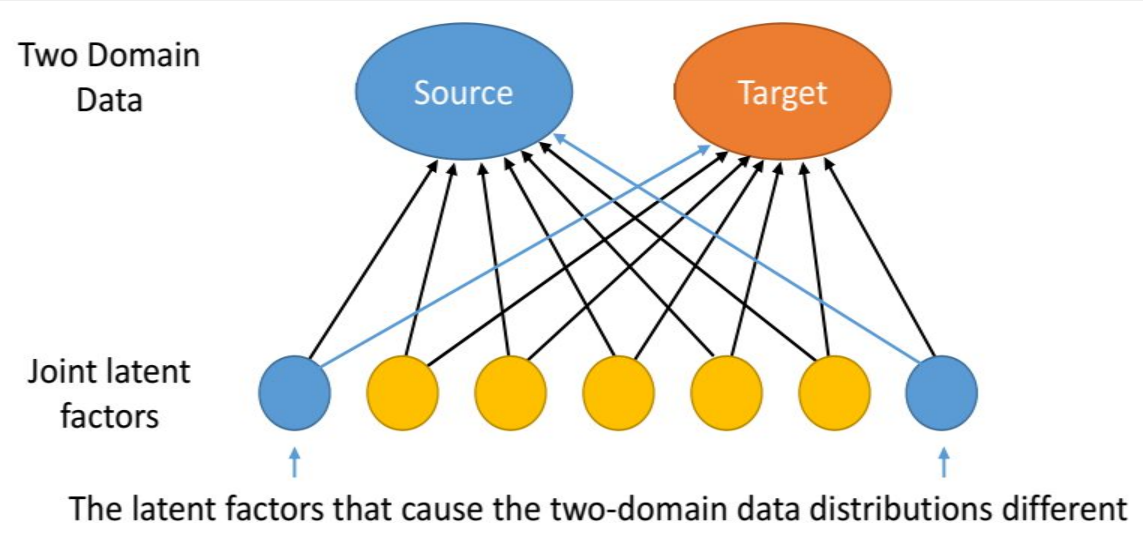
- Ketika domain source dan target **hanya memiliki irisan (overlapping) pada beberapa fitur**
  - Banyak fitur yang hanya mendukung ke salah satunya yaitu domain source atau target

- Solusi yang mungkin:
  - Encode application-specific knowledge
  - Pendekatan umum untuk mempelajari transformasinya φ



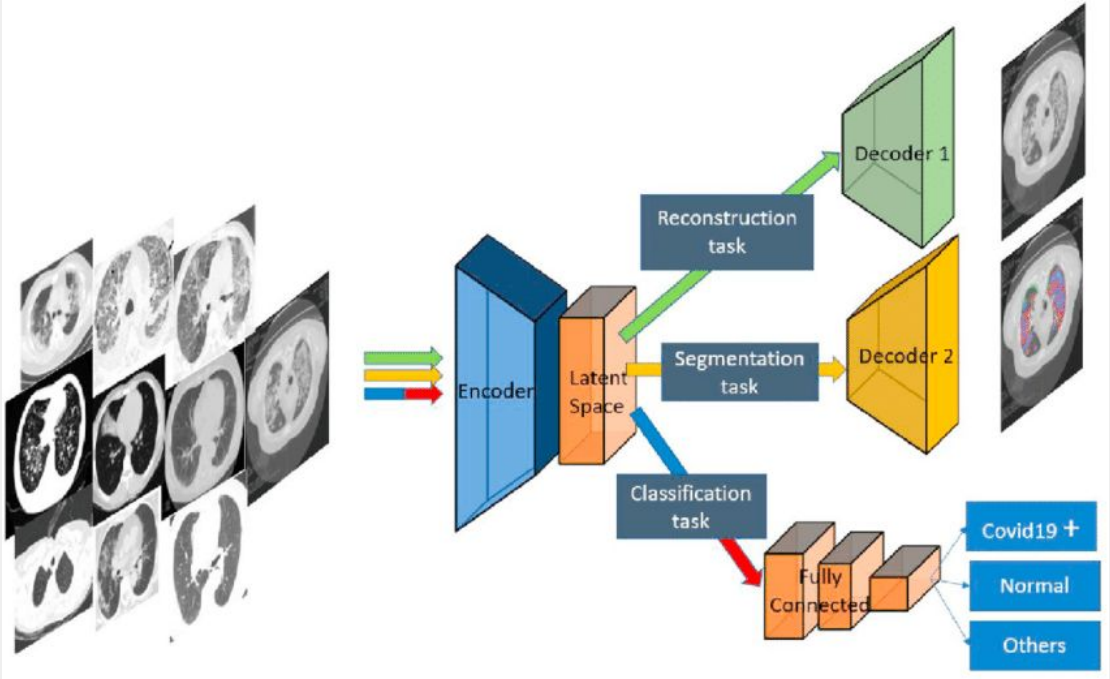
(2) Feature-based Transfer Learning

- Contoh metode yang berbasis pada Feature Transfer Learning
  - Transfer Component Analysis



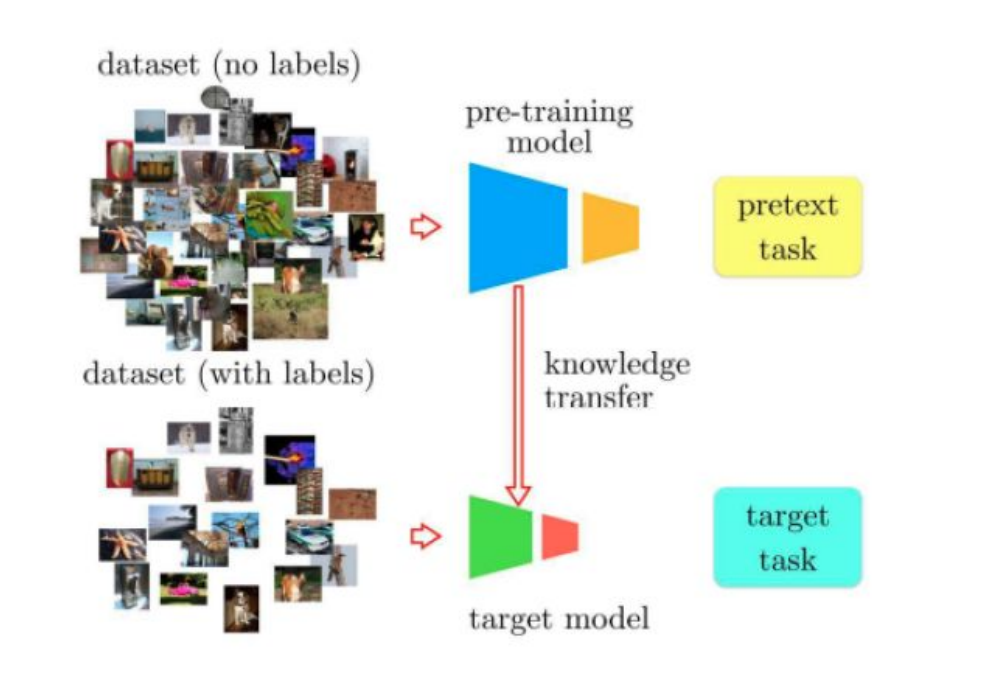
(2) Feature-based Transfer Learning

- Contoh metode yang berbasis pada Feature Transfer Learning
  - Multi-Task Learning



(2) Feature-based Transfer Learning

- Contoh metode yang berbasis pada Feature Transfer Learning
  - Self-Supervised Learning



(3) Parameter-based Transfer Learning

- Fungsi yang berparameter θ, **$f_\theta(x)$ dilatih pada dua domain data**.

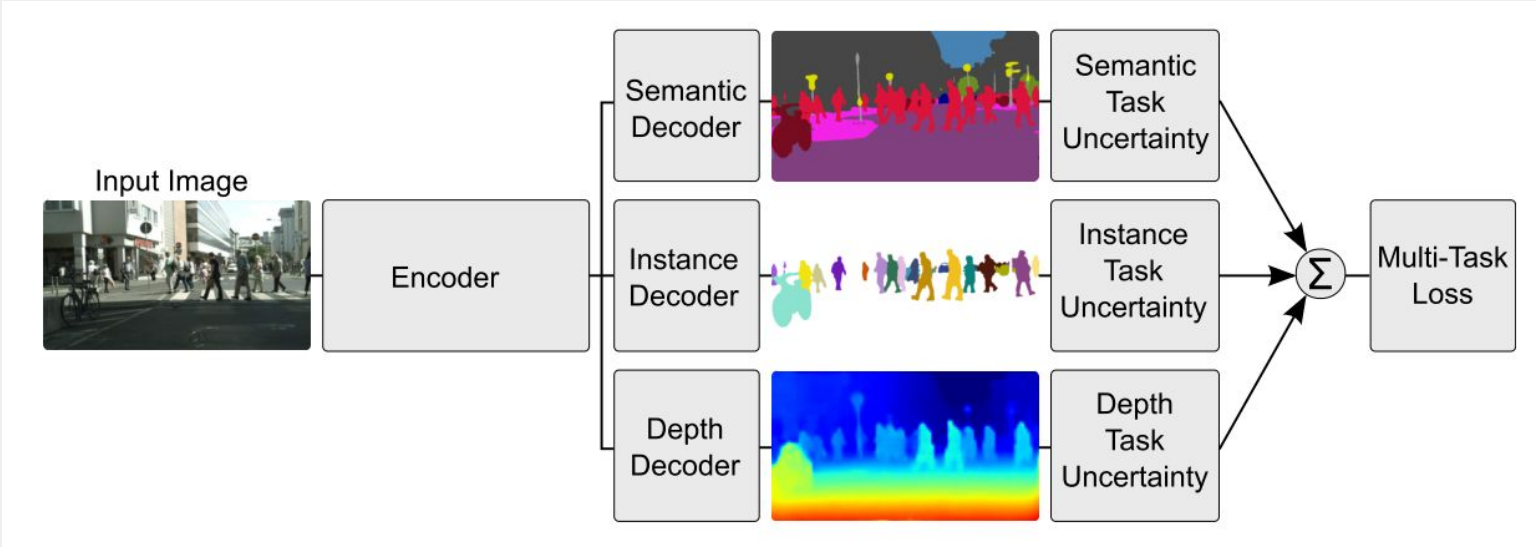
$$\theta_S^* = \arg\min_\theta \sum_{i=1}^{n_S} \mathcal{L}\left(y_{S_i}, f_\theta(x_{S_i})\right) + \lambda\Omega(\theta)$$

$$\theta_T^* = \arg\min_\theta \sum_{i=1}^{n_T} \mathcal{L}\left(y_{T_i}, f_\theta(x_{T_i})\right) + \lambda\Omega(\theta)$$

- Motivasi:
  - $f_{\theta S}(x)$ model yang dilatih dengan baik pada struktur fitur yang banyak pada domain source
  - Jika **dua task memiliki keterhubungan, struktur dari ekstraksi fitur dapat ditransfer untuk melatih $f_{\theta T}(x)$ model pada domain target**

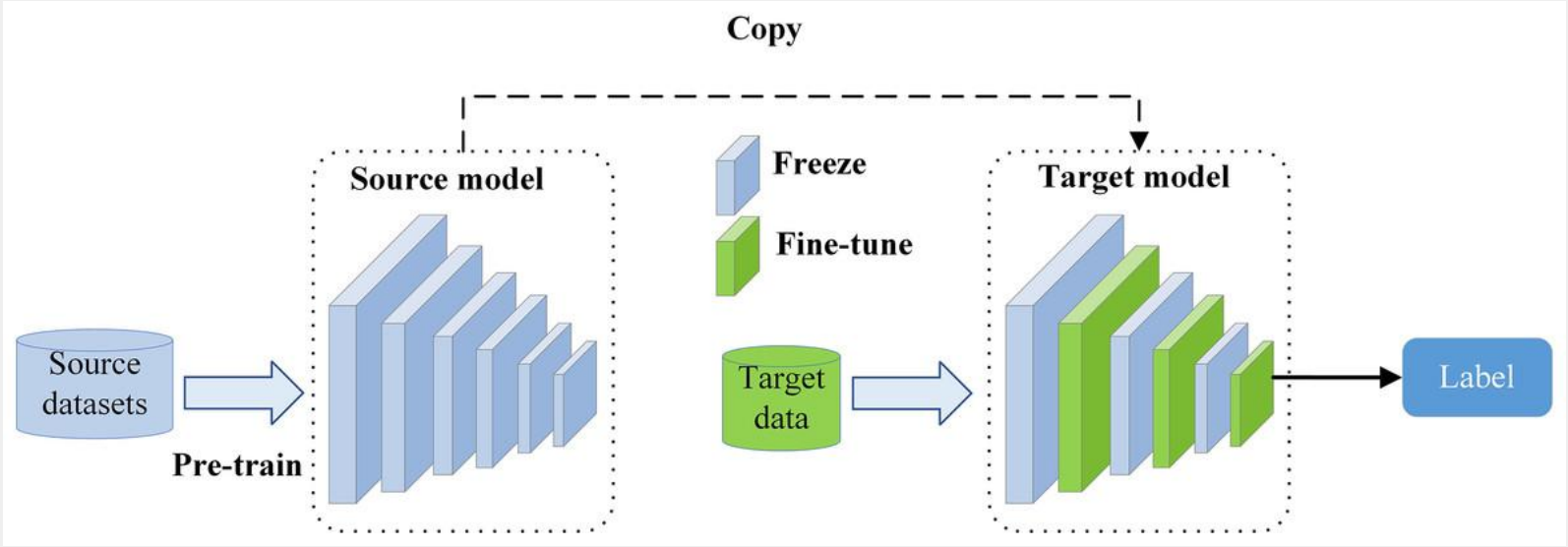
(3) Parameter-based Transfer Learning

- Contoh metode yang berbasis pada Parameter Transfer Learning
  - Multi-Task Learning with Joint Loss Function



(3) Parameter-based Transfer Learning

- Contoh metode yang berbasis pada Parameter Transfer Learning
  - Fine Tuning Model in New Related Domain



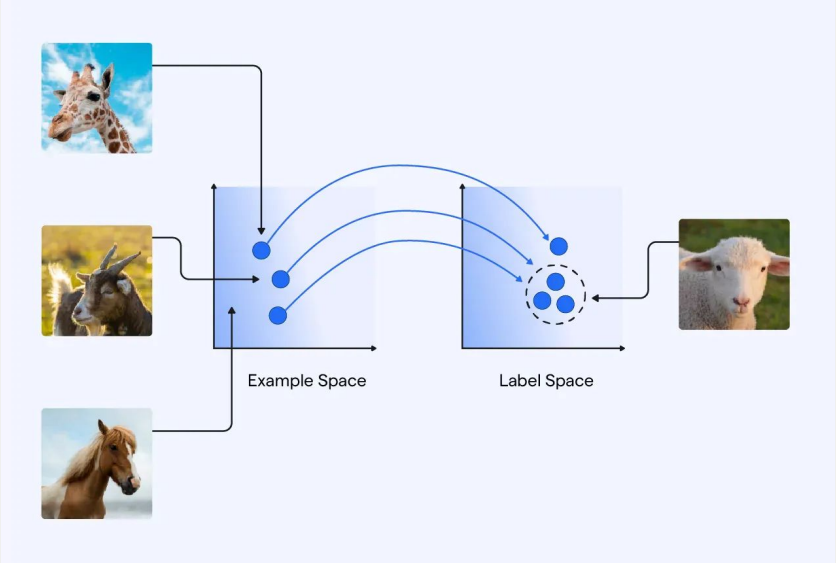
Zero Shot Learning & Fine Tuning



Zero-Shot Learning - Definisi

Zero-shot learning (ZSL) adalah

- Teknik pembelajaran mesin yang memungkinkan model AI untuk mengenali dan mengklasifikasikan objek atau konsep yang belum pernah ditemuinya selama pelatihan,
- dengan **mentransfer pengetahuan dari seen classes ke unseen ones**,
- menggunakan informasi tambahan seperti textual descriptions atau semantic attributes.



Zero-Shot Learning - Cara Kerjanya?

1. **Pre-training on Seen Classes**, model pertama kali dilatih pada kumpulan data berlabel yang besar untuk serangkaian "*seen classes*".
2. **Auxiliary Information**, model juga belajar mengasosiasikan "*seen classes*" dengan atribut atau deskripsi semantik selama pelatihan. Deskripsi-deskripsi ini bertindak sebagai penghubung antara *known and unknown concepts*.
3. **Transferring Knowledge to Unseen Classes**, saat diberikan data dengan "*new unseen classes*" dan tidak ada contoh data yang diberikan selama pelatihan, model menerima deskripsi atau atribut semantiknya.
4. **Generalization**, model tersebut kemudian menggunakan pengetahuan fitur semantik yang dipelajarinya dari "seen classes" untuk menyimpulkan karakteristik kelas baru, yang memungkinkannya melakukan identifikasi meskipun tidak memiliki contoh visual atau berlabel sebelumnya.

Zero-Shot Learning - Mengapa Penting?

- **Reduces Data Labeling Costs:**
  ZSL menghilangkan kebutuhan untuk memberi label secara manual pada kumpulan data besar untuk setiap konsep baru, sehingga menghemat waktu dan biaya secara signifikan.
- **Handles Rare or Novel Categories:**
  Ini sangat berharga dalam skenario dunia nyata seperti mengidentifikasi spesies langka, mendiagnosis penyakit langka, atau mengklasifikasikan jenis produk baru, di mana data berlabel mungkin langka atau tidak ada.
- **Enhances Generalization:**
  ZSL memungkinkan model AI untuk menggeneralisasi pemahaman mereka tentang dunia dan membuat tebakan berdasarkan informasi tentang hal-hal baru yang belum diajarkan secara eksplisit, mirip dengan cara manusia belajar.

Zero-Shot Learning - Penerapannya?

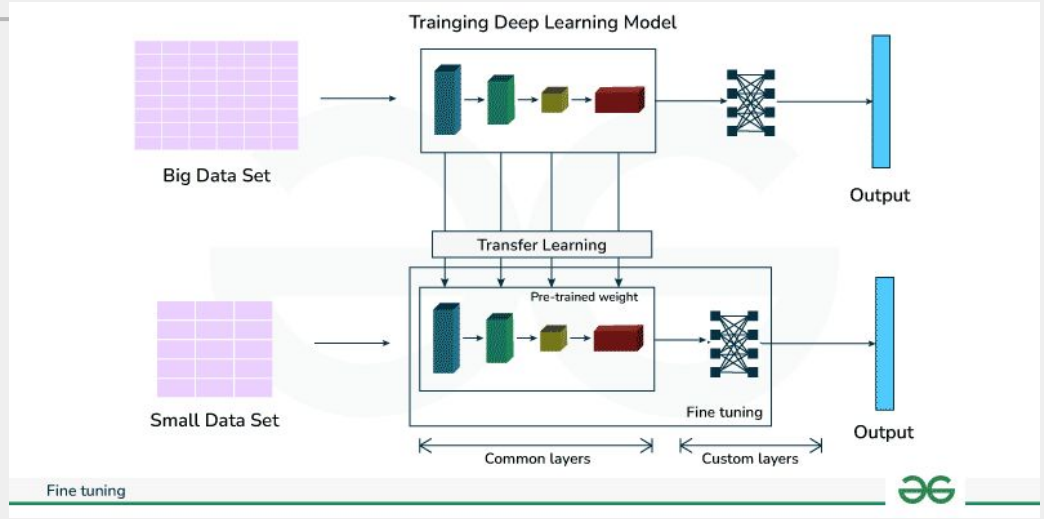
- **Computer vision:**
  Mengidentifikasi hewan langka atau jenis cacat produk baru dalam produksi tanpa data pelatihan sebelumnya.

- **Natural language processing (NLP):**
  - **Text classification**: Secara otomatis mengkategorikan teks ke dalam label baru, seperti pemberian tanda dari jenis baru email yang berisikan penipuan.
  - **Sentiment analysis**: Menentukan sentimen komentar publik terhadap berita terkini tanpa pelatihan khusus untuk suatu peristiwa tertentu.
  - **Chatbots**: Memungkinkan model AI untuk memberikan tanggapan atas pertanyaan baru atau memahami bahasa gaul yang tidak secara eksplisit dilatihkan kepada mereka.

- **Medical diagnostics:**
  Mengidentifikasi kondisi langka atau baru muncul, seperti COVID-19 dalam sinar-X, dengan menghubungkan pola gambar ke deskripsi tekstual dari profesional medis.

Fine Tuning - Definisi

- Fine-tuning adalah teknik di mana model yang telah dilatih sebelumnya akan dilatih lebih lanjut pada set data yang lebih kecil dan spesifik untuk mengadaptasi kemampuan umumnya untuk tugas atau domain tertentu.



- Proses ini menyesuaikan parameter model yang ada untuk meningkatkan kinerjanya pada kasus penggunaan tertentu, sehingga lebih efisien dan efektif untuk penerapan spesifik dibandingkan dengan melatih model dari awal.

Fine Tuning - Cara Kerjanya?

1. **Start with a Pre-trained Model**, model yang telah dilatih pada kumpulan data yang besar dan beragam (model dasar) digunakan sebagai titik awal.
2. **Use a Targeted Dataset**, kumpulan data yang lebih kecil dan berlabel, yang relevan dengan tugas atau domain tertentu digunakan untuk pelatihan lebih lanjut.
3. **Update Model Parameters**, parameter internal model, seperti bobot, sedikit disesuaikan berdasarkan data khusus yang baru.
4. **Adaptation**, tahapan ini memungkinkan model mempertahankan pemahaman umumnya sambil memahami dan mempelajari di area baru yang spesifik.

Fine Tuning - Mengapa Penting?

- **Efficiency**
  Pendekatan ini memerlukan daya komputasi yang jauh lebih sedikit dan titik data berlabel yang lebih sedikit daripada melatih model dari awal, sehingga menjadikannya pendekatan yang lebih efisien.
- **Specialization**
  Hal ini memungkinkan model untuk beradaptasi dengan kasus penggunaan tertentu, seperti skenario bisnis yang berfokus pada layanan pelanggan atau domain ilmiah tertentu.
- **Improved Performance**
  Fine-tuned model dapat mencapai kinerja yang lebih tinggi pada tugas spesifik daripada general-purpose predecessor.
- **Customization**
  Ini membantu model mematuhi format atau pola tertentu yang mengikuti instruksi untuk keluaran yang lebih relevan dan sesuai dengan suatu specific task.

Fine Tuning - Penerapannya?

- **Natural Language Processing (NLP)**
  Suatu perusahaan dapat menyempurnakan general LLM pada manual produk spesifik dengan pertimbangan FAQ atau riwayat percakapan dari customer support.
- **Computer Vision**
  Model AI yang dilatih pada gambar umum dapat disempurnakan menggunakan kumpulan data medical scans, seperti sinar-X atau MRI, untuk mendeteksi penyakit atau anomali tertentu.
- **Smart Farming Specified Task**
  Mengadaptasi model computer vision yang telah dilatih pada data berskala besar, untuk tugas pertanian tertentu, seperti mengidentifikasi dan menyemprot gulma secara tepat.

Terima Kasih ...

Lakukan pembelajaran mandiri dari 3 modul praktek yang telah diberikan.

Persiapkan kelompok Anda untuk Studio Proyek I pada minggu depan (lihat dan pahami spesifikasi tugas kelompoknya).

